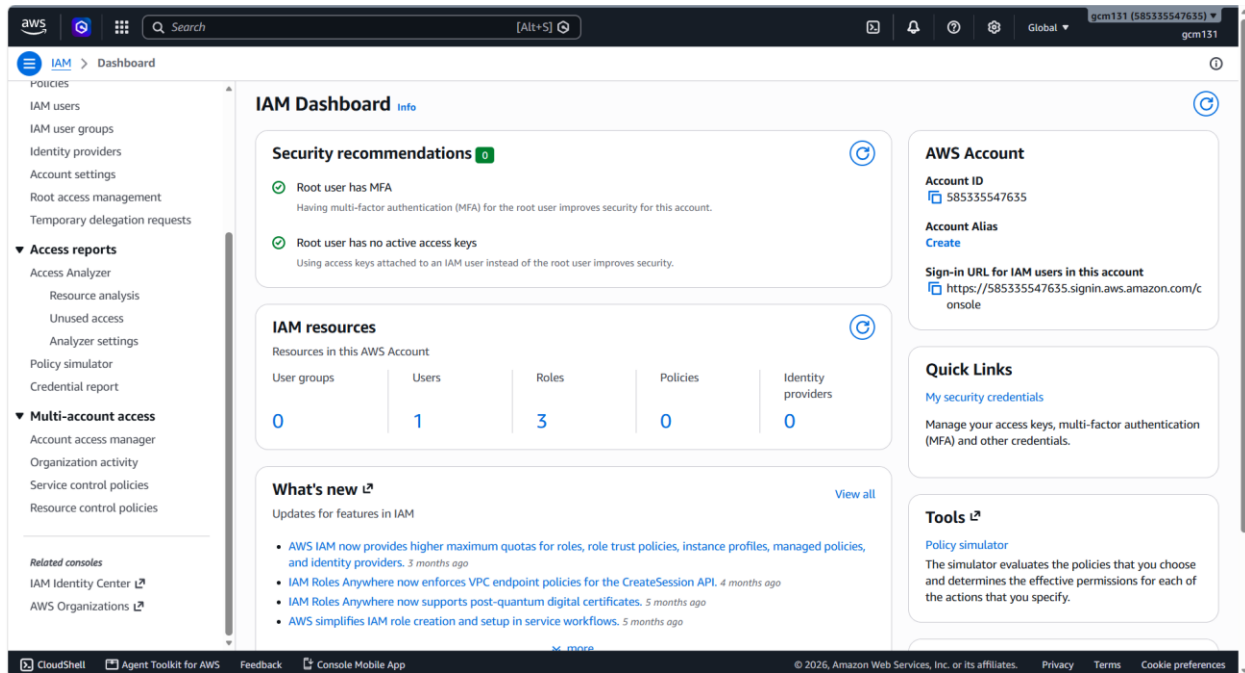


Step 1

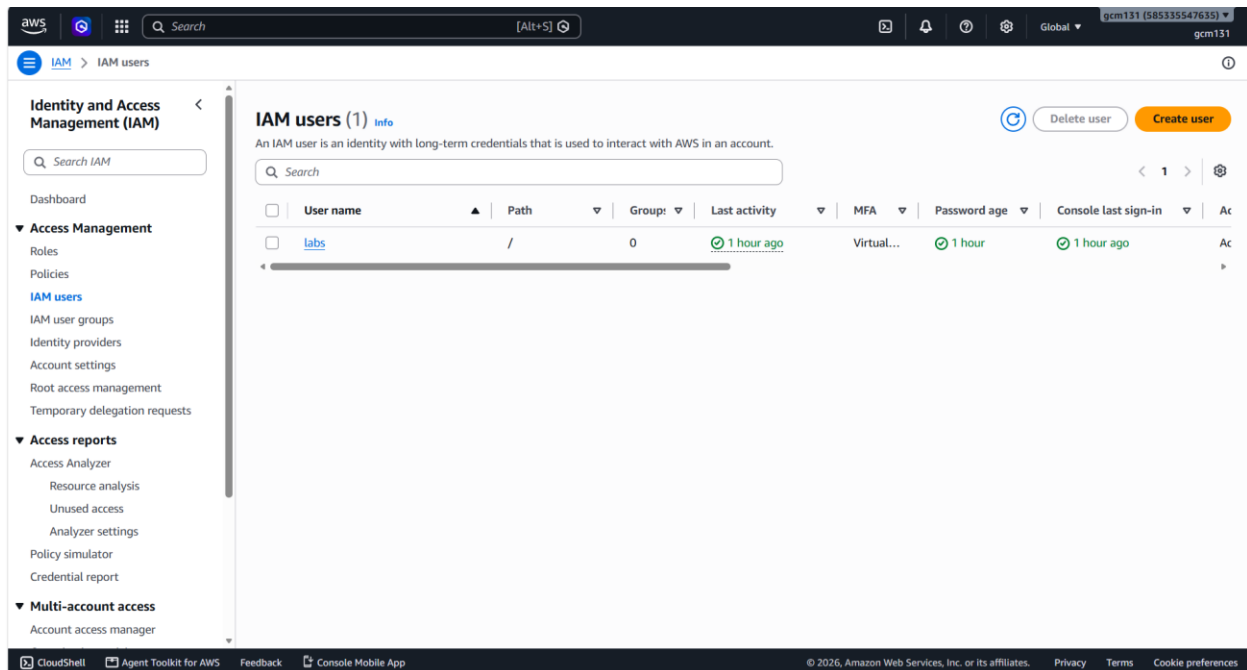
Submission Checklist

Submit a short lab report (1 page) including:

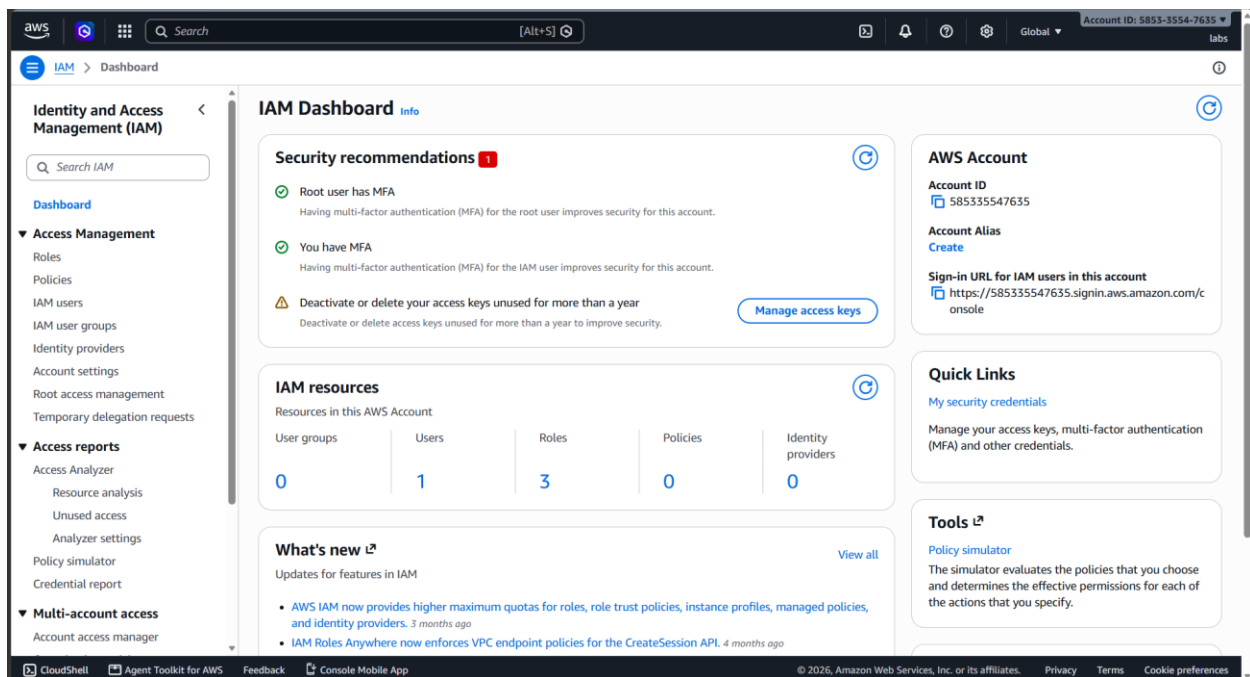
Screenshot of your IAM dashboard showing “MFA on root user: Enabled” (blur/hide any sensitive info).



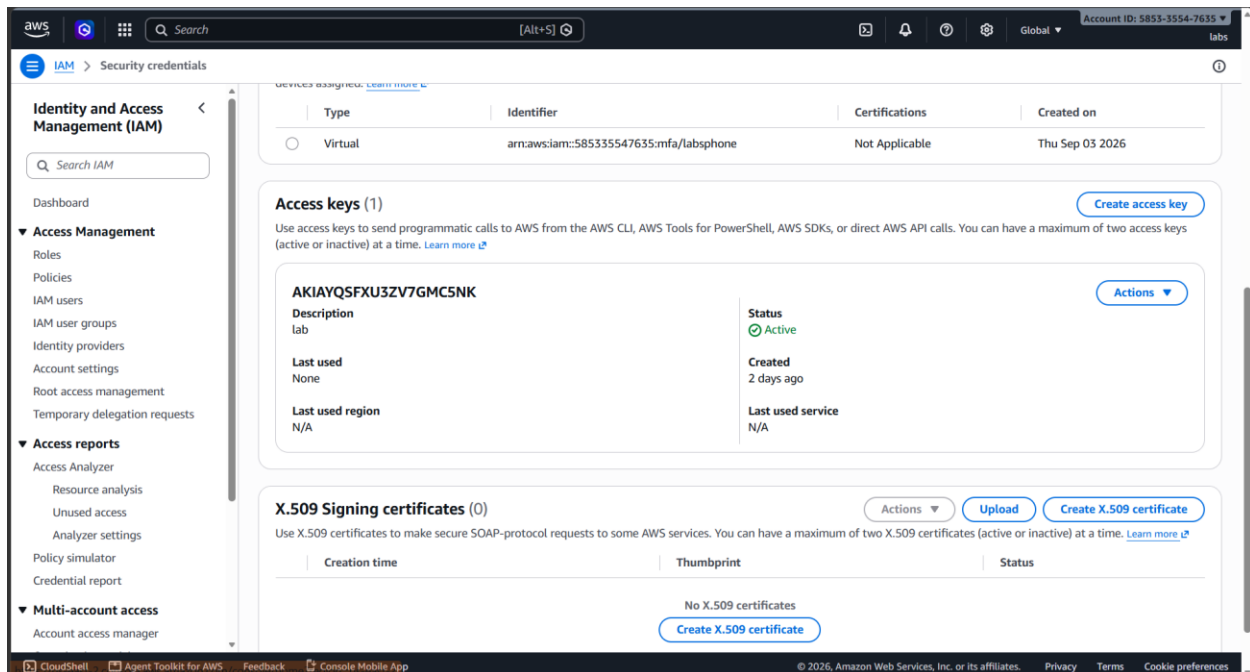
Screenshot of your IAM Users list showing your lab user.



Screenshot showing MFA enabled on the IAM user.



Confirmation (text only — do not include actual key values) that you successfully generated an access key, e.g., “Access key created: AKIA... (last 4 chars: XXXX)”.



One paragraph reflecting on why root user MFA and least-privilege IAM users matter for cloud security.

The root user controls the entire account, this means if the root users password is stolen all data, billing information and everything that the account manages or has access to can be stolen, making MFA a necessary precaution to take. At the same time you should use least-privilege. This means limits every user to only the specific tools they need and means if an attacker breaches a standard account with restricted rights, they cannot jump to sensitive parts of the cloud or lock out the real owner. These steps block wide-scale damage and keep modern cloud systems safe from costly data breaches.

Step 2

Submission Checklist

Submit a short lab report including:

Screenshot of docker --version output.

```
PS C:\Users\user> docker --version
Docker version 29.7.2, build a7dcaa6
PS C:\Users\user>
```

Screenshot of the hello-world container output.

```

PS C:\Users\user> docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Screenshot of cat /etc/os-release run inside the Ubuntu container.

```

root@5c28aba7e18f:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy

```

Screenshot of docker ps -a before and after removing my-ubuntu .

```

PS C:\Users\user> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS          NAMES
5c28aba7e18f   ubuntu:22.04   "/bin/bash"             17 minutes ago Exited (0) 17 seconds ago           my-ubuntu
20235590eaf7   hello-world    "/hello"                19 minutes ago Exited (0) 19 minutes ago           beautiful_bhaskara
ce915deab635   hello-world    "/hello"                2 days ago     Exited (0) 2 days ago           epic_kalam
d32b7c7d73cf   hello-world    "/hello"                2 days ago     Exited (0) 2 days ago           fervent_saha
PS C:\Users\user> docker rm my-ubuntu
my-ubuntu
PS C:\Users\user> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS          NAMES
20235590eaf7   hello-world    "/hello"                20 minutes ago Exited (0) 20 minutes ago           beautiful_bhaskara
ce915deab635   hello-world    "/hello"                2 days ago     Exited (0) 2 days ago           epic_kalam
d32b7c7d73cf   hello-world    "/hello"                2 days ago     Exited (0) 2 days ago           fervent_saha
PS C:\Users\user> |

```

One paragraph explaining, in your own words, the difference between a Docker image and a Docker container.

A docker image and container are like the blueprint and the construction of the blueprint. An image tells docker what OS it is what application to run, dependencies, however this is static and doesn't change. A docker container is the blueprint come to life and is active and running.

Step 3

Submission Checklist

Submit a short lab report including:

Screenshot of aws --version output.

```
root@aa80cd592e15:/# aws --version
aws-cli/2.36.40 Python/3.14.6 Linux/6.6.87.2-microsoft-standard-WSL2 exe/x86_64.ubuntu.22
```

Screenshot of aws sts get-caller-identity showing your IAM username in the ARN (not root).

```
root@aa80cd592e15:/# aws sts get-caller-identity
{
  "UserId": "AIDAYQSFXU3ZUHOVBQKE7",
  "Account": "585335547635",
  "Arn": "arn:aws:iam::585335547635:user/labs"
}
```

Screenshot of git config --global --list showing your name and college email.

```
root@aa80cd592e15:/# git config --global --list
user.name=Gabriel Cerda-Menchaca
user.email=gcerdamenchaca@hawk.illinoistech.edu
```

Screenshot of your GitHub repository showing list_buckets.sh successfully pushed.

The screenshot shows a GitHub repository named 'CloudClass' which is public. The repository has 1 branch (main) and 0 tags. The commit history shows two commits: an initial commit for 'README.md' 7 minutes ago, and a subsequent commit for 'list_buckets.sh' 1 minute ago. The repository is owned by 'Gabriel Cerda-Menchaca'.

File	Commit Message	Time
README.md	Initial commit	7 minutes ago
list_buckets.sh	Add script to list S3 buckets	1 minute ago

One paragraph explaining why you should never commit AWS credentials or GitHub tokens to a repository, even a private one.

You should never commit credentials or tokens to a repository because if the repository is public the credentials will be used and abused by people who shouldn't have access to them in the first place, If the repository is private there is still the risk of something happening to a user that has access either through malware or accidentally making the repo public. The final issue is that git stores a snapshot of every commit thus if a credential is committed you are going to have a hard time removing it.